

Trabalho Prático 3

Algoritmos I

Matheus Soares dos Santos de Freitas

Matrícula: 2024080043

matheussdsf@ufmg.br

1 Introdução

O problema apresentado consiste numa versão do clássico *Maximum Independent Set* (MIS), em que se auxilia o Papai Noel a selecionar o maior grupo possível de duendes que possam trabalhar juntos, uma vez que cada duende possui relações de inimizade com alguns outros. Assim, dado um conjunto de duendes e suas respectivas rivalidades, o objetivo é determinar o maior subconjunto de duendes que não possua conflitos internos. Caso existam múltiplos subconjuntos máximos, deve-se selecionar aquele que seja lexicograficamente menor — isto é, o que apresenta a ordem de índices dos duendes mais “pequena” segundo a ordenação natural. Conhecido por ser NP-difícil, não é esperado que exista uma solução polinomial para esse problema em grafos gerais. Entretanto, como a quantidade de duendes N é limitada a no máximo 40, é possível empregar técnicas otimizadas de força bruta, combinando bitmasks com *bottom-up* (programação dinâmica com tabulação) e o método *meet-in-the-middle* (MITM), tornando o tempo de execução viável.

2 Modelagem

Para lidar com esse problema de alta complexidade combinatória, foi empregada uma modelagem que utiliza uma classe dedicada a representar o grafo, bem como estruturas de dados otimizadas baseadas em máscaras de bits.

2.1 Classes e Estruturas de Dados

Classe `poloNorte`

A classe `poloNorte` encapsula toda a representação do grafo e das operações necessárias para a seleção da equipe. Seus principais componentes são:

- **`conflitosDuendes`**: vetor de máscaras de 64 bits que representa a adjacência global dos duendes. Para cada duende i , a máscara correspondente possui bits ativados indicando os identificadores dos duendes que entram em conflito com i .
- **`conflitosEsquerda` e `conflitosDireita`**: vetores contendo, respectivamente, as máscaras de conflito internas aos subconjuntos esquerdo e direito do particionamento utilizado na etapa *meet-in-the-middle*.
- **`conflitosEsquerdaParaDireita` e `conflitosDireitaParaEsquerda`**: vetores que armazenam as máscaras de conflitos cruzados entre os dois grupos, isto é, conflitos entre duendes do lado esquerdo e do lado direito.

Além disso, a classe fornece os seguintes métodos auxiliares:

- **`mascaraParaVetor`**: converte uma máscara de bits em um vetor contendo os índices globais dos duendes selecionados.
- **`vetorLexMenor`**: compara duas listas de identificadores segundo a ordem lexicográfica e retorna um valor booleano indicando qual delas é menor.
- **Construtor `poloNorte`**: recebe o número total de duendes e a lista de conflitos, inicializando todas as estruturas de adjacência, inclusive a separação entre subconjuntos esquerdo e direito.

2.2 Algoritmo para Construção da Equipe

O método `formarEquipe` implementa a estratégia de resolução, e é o algoritmo de destaque da solução, retornando os identificadores globais dos duendes que compõem o maior conjunto independente possível - com adoção do critério lexicográfico em caso de empate. A solução faz uso de *bitmasks* e combina uma abordagem *meet-in-the-middle* com uma estratégia de tabulação (bottom-up) sobre as combinações do grupo da direita - para evitar recálculos -, permitindo selecionar, de forma eficiente, a melhor composição conjunta entre os dois grupos.

Solução

A função `formarEquipe` resolve o problema aplicando uma estratégia de três etapas:

2.3 Etapa 1: Metade esquerda.

A função enumera todas as máscaras possíveis da esquerda, verificando independência por meio de `conflitosEsquerda`. Cada máscara válida é armazenada em `maskasIndependentesEsquerda`. A verificação utiliza operações de bitmask e `__builtin_ctz` para isolar vértices e testar adjacências de forma eficiente.

2.4 Etapa 2: Metade direita e programação dinâmica sobre submáscaras (bottom-up).

Primeiro, em loop, `formarEquipe` percorre todas as máscaras da direita e identifica as independentes usando `conflitosDireita`, preenchendo as tabulações `tamanhoEquipes` e `equipes`. Em seguida, cada máscara é atualizada pela melhor entre ela e suas submáscaras (removendo um único duende), substituindo-a caso a submáscara seja maior ou, em empate, lexicograficamente menor. Essa etapa garante que a tabela `equipes` sempre represente o maior conjunto independente possível contido em cada máscara.

2.5 Etapa 3: Combinação final.

Para cada máscara independente da esquerda, obtém-se o conjunto de vértices bloqueados na direita via `conflitosEsquerdaParaDireita`. A máscara de vértices permitidos é então usada como índice direto em `equipes` para recuperar instantaneamente a melhor equipe da direita. As duas partes são convertidas em vetores por `mascaraParaVetor`, intercaladas em ordem crescente e comparadas por tamanho e, em caso de empate, por meio de `vetorLexMenor`. A melhor combinação é atualizada ao longo da iteração.

Resultado: Ao final, `formarEquipe` retorna o maior conjunto independente possível, escolhendo o vetor lexicograficamente mínimo entre soluções de mesmo tamanho, explorando a decomposição em duas metades e a DP sobre submáscaras para maximizar eficiência.

Análise de Complexidade

A função `formarEquipe` divide o conjunto total de duendes em duas metades: n_L (esquerda) e n_R (direita), com $n = n_L + n_R$ e $n_L \approx n_R \approx n/2$. As três etapas executadas dentro da própria função têm as seguintes complexidades:

Etapa 1: Conjuntos independentes da esquerda

A função testa todas as 2^{n_L} máscaras da esquerda, percorrendo os bits ativos de cada máscara e verificando conflitos.

$$T_{\text{esq}} = O(2^{n_L} \cdot n_L), \quad M_{\text{esq}} = O(2^{n_L})$$

Etapa 2: Metade direita e PD por submáscaras na direita

Primeiro, todas as 2^{n_R} máscaras da direita são testadas por independência com custo $O(n_R)$ cada. Em seguida, o processo de correção de submáscaras aplica tabulação (bottom-up) em todas as máscaras, percorrendo cada bit.

$$T_{\text{dir}} = O(2^{n_R} \cdot n_R), \quad M_{\text{dir}} = O(2^{n_R})$$

Etapa 3: Combinação final

Para cada máscara independente da esquerda (no máximo 2^{n_L}): conversão para vetor: $O(n_L)$; construção da máscara de bloqueios: $O(n_L)$; consulta ao melhor subconjunto da direita: $O(1)$; fusão ordenada dos vetores: $O(n)$.

Logo,

$$T_{\text{comb}} = O(2^{n_L} \cdot n)$$

Tempo Total

Somando todas as etapas da própria função `formarEquipe()`:

$$T(n) = O(2^{n_L} n_L) + O(2^{n_R} n_R) + O(2^{n_L} n)$$

Assumindo $n_L \approx n_R \approx n/2$:

$$T(n) = O(n \cdot 2^{n/2})$$

Memória Total

O consumo de memória é dominado pelo armazenamento das máscaras válidas da esquerda e pela tabela de PD da direita:

$$M(n) = O(2^{n/2})$$

3 Considerações Finais

A solução implementada conseguiu respeitar todas as restrições do problema temático natalino: identificar o maior grupo de duendes que não possuem rivalidades, garantindo também que a escolha final seja a lexicograficamente menor possível. Apesar do problema ser intrinsecamente difícil, o uso de *bitmasks* e *meet-in-the-middle* permitiu reduzir o espaço de busca de forma significativa, possibilitando resolver instâncias com até 40 duendes de forma eficiente. Já a tabulação (*bottom-up*) para a metade direita reduziu os custos de recálculo das combinações.

A parte mais simples do desenvolvimento foi a criação das máscaras de adjacência para registrar os conflitos entre os duendes. A parte mais trabalhosa, por outro lado, foi garantir corretamente as condições lexicográficas na união dos subconjuntos e otimizar as verificações de compatibilidade entre as duas metades.

Ao final, o método empregado mostrou-se adequado ao contexto narrativo do trabalho, eficiente para o tamanho das instâncias e alinhado às melhores práticas para resolver o problema de conjunto independente máximo em grafos pequenos.

4 Referências

- Cormen, T., Leiserson, C., Rivest, R., Stein, C. *Introduction to Algorithms*. MIT Press.
- Kleinberg, J., Tardos, É. *Algorithm Design*. Pearson.
- Garey, M., Johnson, D. *Computers and Intractability: A Guide to the Theory of NP-Completeness*.